

Allocare la Memoria per il Kernel

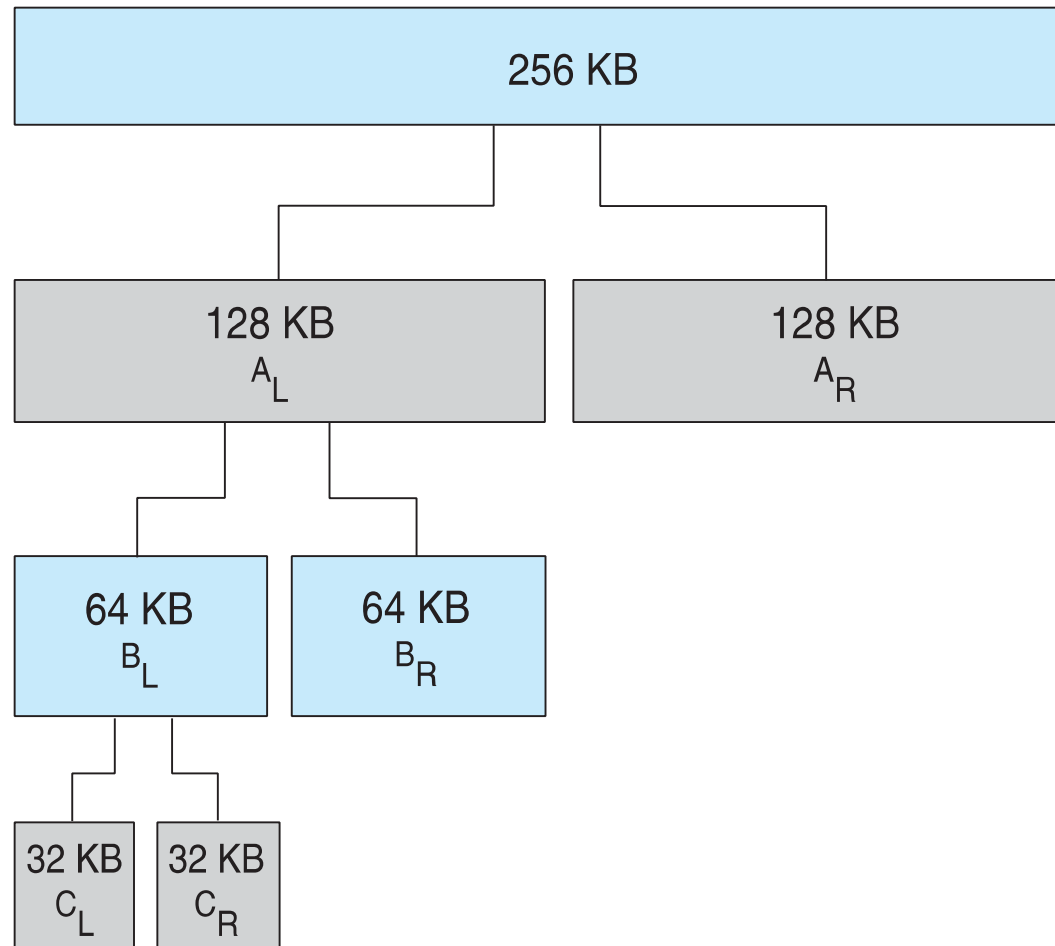
- Trattata in modo differente dalla memoria per processi utente
- Spesso allocate memoria da un pool di free-memory differente
- Due sono le ragioni principali:
 - Kernel richiede memoria per strutture di varie dimensioni, sotto la dimensione della pagina, deve usare la memoria in modo più conservative e mirato
 - La memoria del Kernel in alcuni casi necessita di essere contigua
 - ▶ I.e., dispositivi I/O che dialogano direttamente in memoria fisica, in questi casi la memoria deve essere contigua
- Di seguito si presentano due metodi per l'allocazione di memoria ai processi kernel:
 - Buddy System
 - Allocazione slab

Sistema Buddy

- Alloca memoria da segmenti di dimensioni fissate che consistono di pagine fisicamente contigue
 - Un segmento largo e contiguo suddiviso in porzioni più piccolo; più segmenti combinati rapidamente per ottenere frammento contiguo
- Memoria allocata usando **allocatore di potenza-di-2**
 - Soddisfa richieste in unità di dimensione potenza di 2
 - Richieste approssimate alla più alta Potenza di 2
 - Quando occorre un'allocazione più piccolo della disponibile, il chunk corrente diviso in due buddies della prossima potenza di 2 più bassa
 - ▶ Continua finché un chunk appropriato non è disponibile
- Per esempio, assume chunk di 256KB disponibile, il kernel richiede 21KB
 - Diviso in A_L e A_R di 128KB ognuno
 - ▶ Uno ulteriormente diviso in B_L e B_R di 64KB
 - Uno ulteriormente in C_L e C_R di 32KB ognuno – uno usato per soddisfare la richiesta

Buddy System Allocator

physically contiguous pages

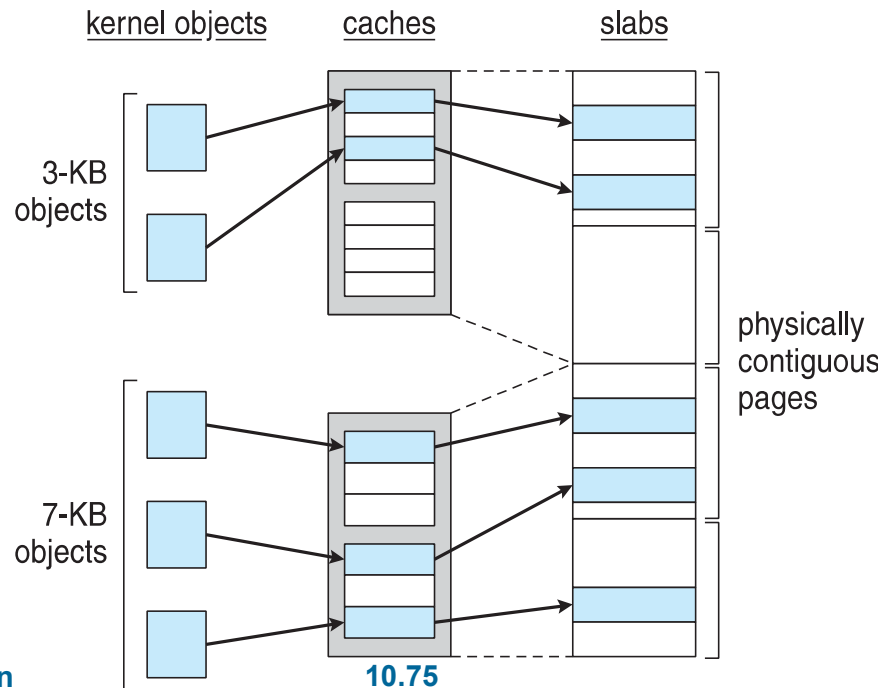


Sistema Buddy

- ❑ Alloca memoria da segmenti di dimensioni fissate che consistono di pagine fisicamente contigue
 - ❑ Un segmento largo e contiguo suddiviso in porzioni più piccolo; più segmenti combinati rapidamente per ottenere frammento contiguo
- ❑ Memoria allocata usando **allocatore di potenza-di-2**
 - ❑ Soddisfa richieste in unità di dimensione potenza di 2
 - ❑ Richieste approssimate alla più alta Potenza di 2
 - ❑ Quando occorre un'allocazione più piccolo della disponibile, il chunk corrente diviso in due buddies della prossima potenza di 2 più bassa
 - ▶ Continua finché un chunk appropriato non è disponibile
- ❑ Vantaggio – velocemente **compatta** chunk non usati in chunk più grandi
- ❑ Svantaggio – frammentazione interna
 - ❑ Sempre potenze di due, e la memoria richiesta può essere minore

Slab Allocator

- ❑ Strategia che elimina frammentazione interna
 - ❑ Oggetti di un certo tipo preallocati
 - ❑ Riutilizzo di oggetti dello stesso tipo
- ❑ **Slab** fatto da una o più pagine fisicamente contigue
 - ❑ È il contenitore di oggetti di un certo tipo
- ❑ Una **cache** consiste di uno o più slab
 - ❑ Una cache per per ogni data structure del kernel
 - ❑ Ogni cache popolata di **oggetti** – istanze delle strutture dati



Slab Allocator

- ❑ Strategia che elimina frammentazione interna
 - ❑ Oggetti di un certo tipo preallocati
 - ❑ Riutilizzo di oggetti dello stesso tipo
- ❑ **Slab** fatto da una o più pagine fisicamente contigue
 - ❑ È il contenitore di oggetti di un certo tipo
- ❑ Una **cache** consiste di uno o più slab
 - ❑ Una cache per per ogni data structure del kernel
 - ❑ Ogni cache popolata di **oggetti** – istanze delle strutture dati
- ❑ Quando è creata la cache riempita di oggetti marcati come **free**
- ❑ Quando le strutture sono memorizzate oggetti marcati come **used**
- ❑ Benefici: non frammentazione, veloce soddisfazione di richieste di memoria

Slab Allocator in Linux

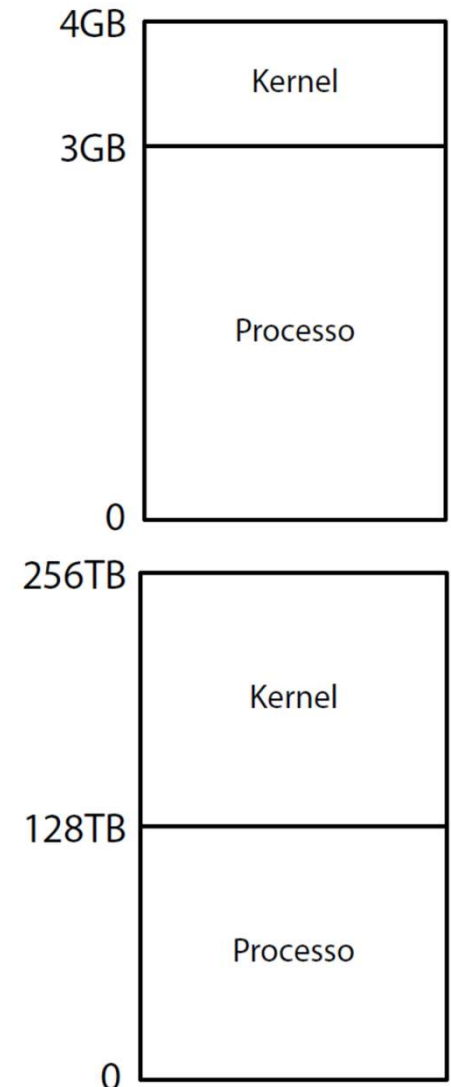
- ❑ Esempio: per process descriptor è di tipo `struct task_struct`
- ❑ Appross. 1.7KB di memoria
- ❑ Nuovo task -> alloca new struct dalla cache
 - ❑ Userà `free struct task_struct`
- ❑ Slab può essere in tre possibili stati
 1. Full – tutto usato
 2. Empty – tutto libero
 3. Partial – mix di free e used
- ❑ Su richiesta, slab allocator
 1. Usa free struct in partial slab
 2. Se non c'è, prende uno degli empty slab
 3. Se non c'è empty slab, crea un nuovo empty slab

Slab Allocator in Linux

- ❑ Slab iniziato in Solaris, ora diffuso sia per Kernel mode e memoria user in molti SO, in Linux da 2.2, prima Buddy
- ❑ Linux 2.2 aveva SLAB, ora ha allocatori SLOB e SLUB
 - ❑ SLOB per sistemi con limitata memoria
 - ▶ Simple List Of Blocks – mantiene 3 liste di oggetti per piccoli, medi e grandi
 - ▶ Oggetti 256 byte, 1024 byte, più di una pagina
 - ▶ First fit-policy per l'allocazione
 - ❑ SLUB è uno SLAB ottimizzato,
 - ▶ Introdotto da 2.6.24 per sostituire SLAB
 - ▶ Migliore utilizzo dei processori multipli

Layout memoria x86

- Nell'architettura x86-32 lo spazio degli indirizzi lineari è diviso in due aree
 - **0-3GB.** Assegnato ad ogni processo (sostituito con context-switch).
 - **3-4GB.** Assegnato al kernel, non sostituito
- Nell'architettura x86-64 lo spazio di indirizzi diviso
 - **0-128TB.** Assegnato ad ogni processo.
 - ▶ Sostituito ad ogni cambio contesto
 - **128TB-256TB.** Assegnato al kernel, non sostituito.



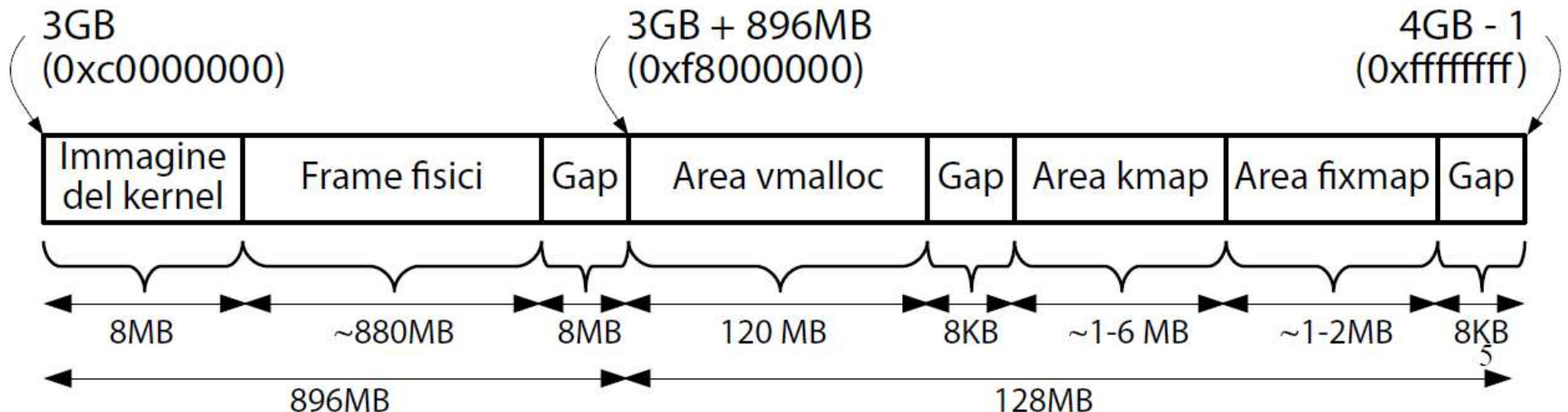
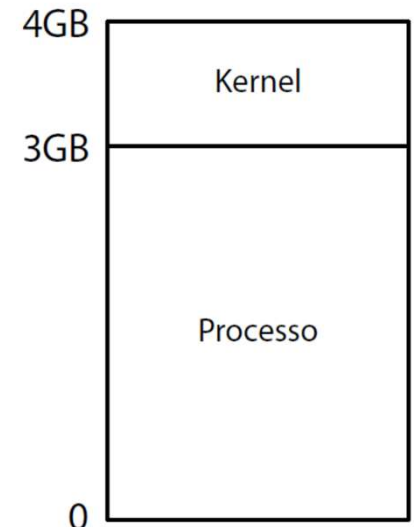
Layout memoria x86

- Nell'architettura x86-32 lo spazio degli indirizzi lineari è diviso in due aree
 - **0-3GB.** Assegnato ad ogni processo (sostituito con context-switch).
 - **3-4GB.** Assegnato al kernel, non sostituito

I primi 8MB di memoria dedicati all'**immagine del kernel**.
Decompresso il codice del kernel all'inizio del boot. Contigui

I successivi 880MB sono dai frame fisici. Possono essere liberi o allocati (per oggetti del kernel). Contigui fisicamente.

- Frame liberi: gestiti dall'allocatore buddy.
- Frame occupati: gestiti dall'allocatore SLAB (kmalloc())

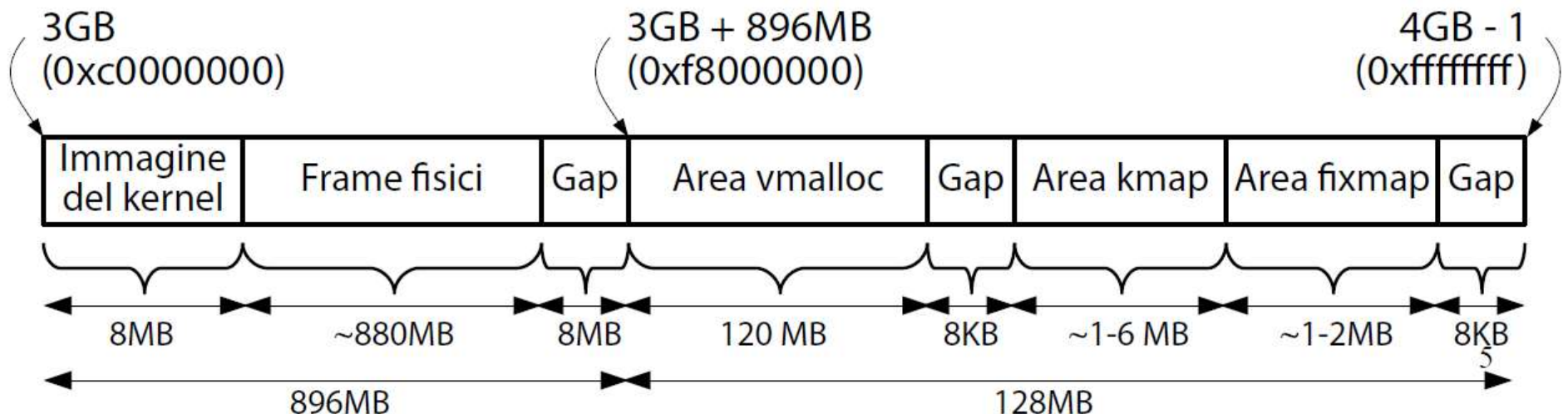
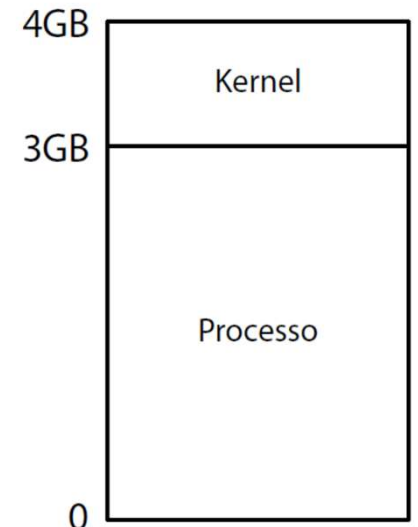


Layout memoria x86

- Nell'architettura x86-32 lo spazio degli indirizzi lineari è diviso in due aree
 - **0-3GB.** Assegnato ad ogni processo (sostituito con context-switch).
 - **3-4GB.** Assegnato al kernel, non sostituito

L'area vmalloc è grande circa 120MB ed è dedicata alle porzioni di codice e dati dei moduli del kernel. I moduli sono attivati e disattivati più volte durante il ciclo di vita del kernel, l'area di memoria successiva non è garantita fisicamente contigua.

- vmalloc(): allocazione blocco.
- vfree(): deallocazione blocco.



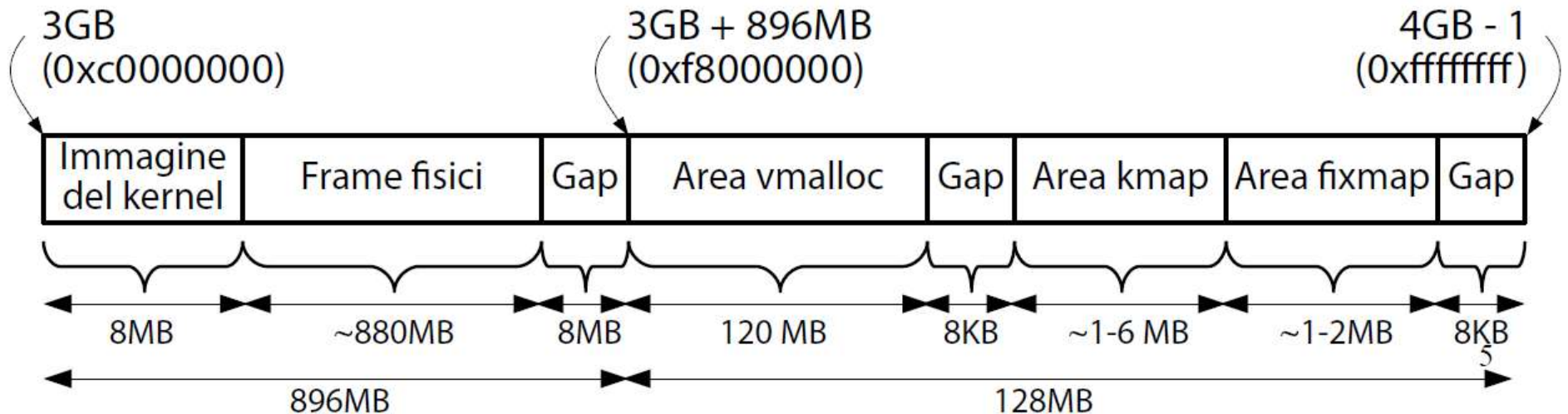
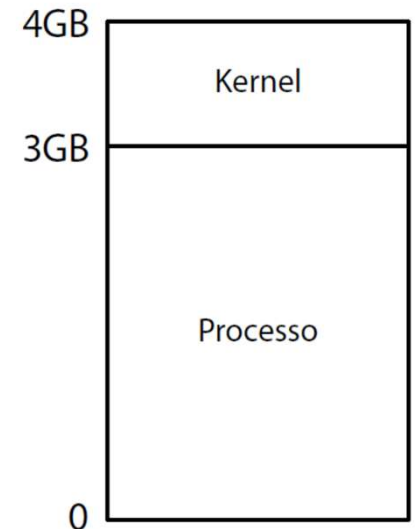
Layout memoria x86

- Nell'architettura x86-32 lo spazio degli indirizzi lineari è diviso in due aree
 - **0-3GB.** Assegnato ad ogni processo (sostituito con context-switch).
 - **3-4GB.** Assegnato al kernel, non sostituito

L'area kmap, grande circa 1-6MB è dedicata alla creazione di mappature lineari verso frame fisici oltre il GB. Tali mapping sono di breve durata (es. operazioni di I/O)

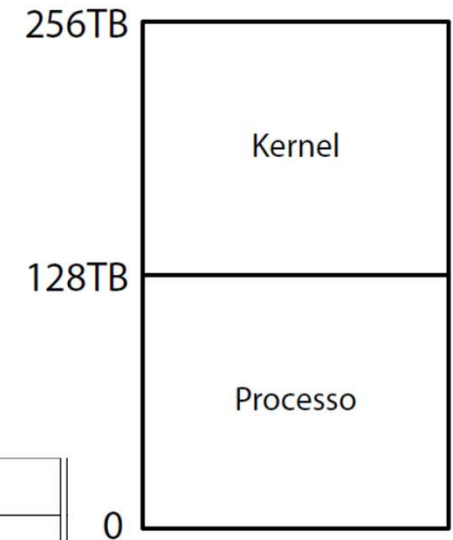
L'area fixed map contiene pagine di 4KB i cui indirizzi lineari iniziali sono costanti (gestibili dal loader dinamico)

- set_fixmap(idx, phys)
- clear_fixmap(idx, phys)



Layout memoria x86-64

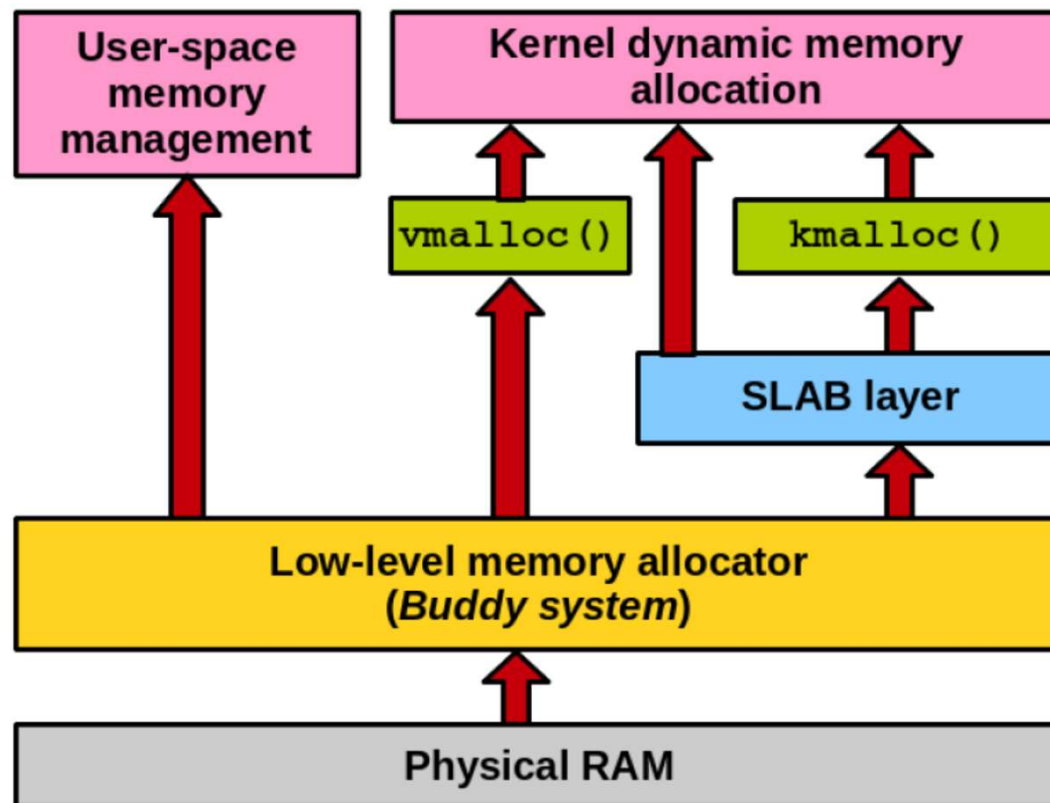
- Nell'architettura x86-64 lo spazio di indirizzi diviso
 - **0-128TB.** Assegnato ad ogni processo.
 - ▶ Sostituito ad ogni cambio contesto
 - **128TB-256TB.** Assegnato al kernel, non sostituito.



Indirizzo iniziale	Indirizzo finale	Dimensione	Scopo
0xffff800000000000	0xffff80fffffffffff	1TB	Gap
0xffff810000000000	0xffff87ff00000000	16EB	Spazio non indirizzabile
0xffff880000000000	0xffffc7fffffffffff	64TB	Frame fisici
0xffffc80000000000	0xffffc8fffffffffff	1TB	Gap
0xffffc90000000000	0xffffe8fffffffffff	32TB	Area vmalloc (I/O)
0xffffe90000000000	0xffffe9fffffffffff	1TB	Gap
0xffffea0000000000	0xffffeafffffffffffff	1TB	Area fixmap
0xffffeb0000000000	0xfffffff7fffffff	21TB	Non usato
0xffffffff80000000	0xfffffffffa000000	512MB	Immagine del kernel
0xfffffffffa000000	0xfffffffffff5ffff	1525MB	Area moduli kernel
0xfffffffffff60000	0xfffffffffffdffff	8MB	Area vsyscall
0xfffffffffffe0000	0xffffffffffffffff	2MB	Non usato

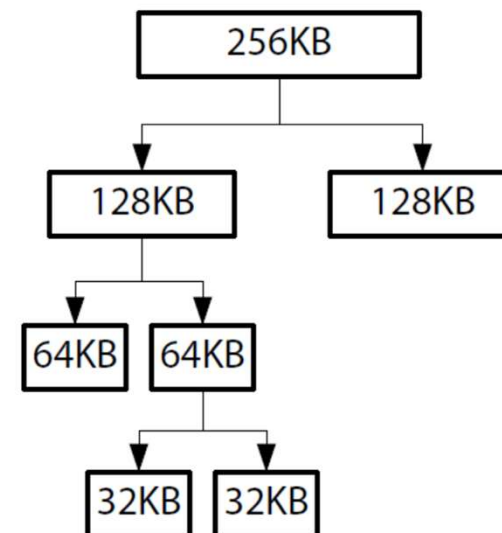
Gestione Frame

□ Gestione memoria basso livello



Gestione Frame Liberi

- ❑ La memoria allocata dal kernel (e dai processi) deve essere supportata dai frame fisici
- ❑ Occorre gestione efficiente della lista di frame liberi
 - ❑ Allocazione: si prendono frame dalla lista
 - ❑ Deallocazione: si rilasciano frame alla lista
- ❑ Lista collegata di `page_struct` non fattibile
- ❑ Il kernel gestisce l'area dei frame liberi mediante il Sistema Buddy
- ❑ L'allocatore buddy è buono per gestire grandi aree di memoria libera



Gestione Frame Occupati

- ❑ La memoria allocata dal kernel (e dai processi) deve essere supportata dai frame fisici
- ❑ L'allocatore a lastre (SLAB allocator) inizialmente implementato in Solaris 2.4 (1994) gestisce “cache” di porzioni di memoria
 - ❑ preallocate.
 - ❑ fisicamente contigue.
 - ❑ linearmente contigue.
- ❑ Quando il kernel chiede una porzione di memoria l'allocatore fornisce un'area preallocata dalla cache (quando la memoria è liberata, restituita alla cache)
- ❑ Uno SLAB è un insieme di pagine logiche associate a frame fisici contigui
- ❑ Uno o più SLAB contigui formano una cache:
 - ❑ Ciascuna cache è un contenitore di oggetti di un dato tipo
 - ❑ Un oggetto è un'area di memoria di una specifica dimensione
 - ❑ Funzioni: `kmem_cache_create()`, `kmem_cache_alloc()`, `kmem_cache_free()`
 - ▶ Anche `kmem_cache_grow()`, `kmem_cache_shrink()`, `kmalloc()`, `kfree()`
 - ▶ Allocatore buddy invocato per ottenere memoria contigua

Altre Considerazioni -- Prepaging

□ Prepaging

- Pure demand paging crea molti page fault allo start di un processo
- Occorre ridurre il numero di page fault allo start-up del processo
- Preallocare alcune delle pagine che un processo userà prima che siano riferite
- Se si utilizza il modello Working Set, si può usare WS per ripristinare le pagine quando il processo è richiamato in memoria

□ Costo/beneficio del prepaging

- Se pagine prepaged non usate, I/O e memoria “sprecate”
- Assumi s pagine sono prepaged con percentuale α delle pagine usata
 - ▶ costo di $s * \alpha$ pages fault evitati $>$ o $<$ del costo del prepaging $s * (1 - \alpha)$ pagine non necessarie?
 - ▶ α vicino a zero $\Rightarrow$ prepaging perde
- Più facile fare prepaging di un file che di un programma eseguibile

Altre Considerazioni – Page Size

- ❑ I progettisti di SO raramente possono decidere la dimensione delle pagine
- ❑ Dimensione definite in fase di progetto
 - ❑ Sempre potenza di 2, di solito nel range 2^{12} (4,096 bytes) fino a 2^{22} (4,194,304 bytes)
- ❑ Selezione della dimensione delle pagine deve tenere in considerazione:
 - ❑ Frammentazione
 - ❑ Dimensione della page table
 - ▶ Per VM di 4MB (2^{22}) si hanno 4096 pagine di 1024 byte e 512 da 8192 byte
 - ❑ Risoluzione
 - ▶ Si accede alla porzione di memoria necessaria
 - ▶ Minore I/O overhead
 - ▶ Maggior numero di page faults
- ❑ Pagine piccolo: località, meno frammentazione, meno I/O overhead
- ❑ Pagine grandi: meno page fault, page table più piccolo

Altre Considerazioni – TLB Reach

- Hit ratio della TLB – quantità di indirizzi risolti in TLB
 - Per aumentare si possono introdurre TLB con più registry, ma costoso
- **TLB Reach** - la quantità di memoria accessibile dalla TLB
 - $TLB\ Reach = (TLB\ Size) \times (Page\ Size)$
- Idealmente, il working set di ogni processo va immagazzinato nella TLB
 - altrimenti c'è alto livello di page fault ...
 - Per aumentare il reach incrementa la dimensione di pagina (es. da 4 a 16 KB)
 - ... ma potrebbe portare ad un incremento della frammentazione interna dal momento che non tutte le applicazioni richiedono dimensioni grandi
 - ... oppure fornire dimensioni di pagina multiple
 - Permette alle applicazioni di usare pagine grandi senza incorrere in problemi di frammentazione
- Es. Linux ha pagine di 4KB, ma anche più grandi, e.g., 2M
- Es. ARMv8 più tipi di pagine,
 - ▶ TLB entry hanno contiguous bit (riferiscono blocchi di memoria contigua)
 - ▶ es., da 64KB (16x4KB), 1GB (32x32MB), 2MB (32x64KB)

Altre Considerazioni – Tab Pagine inverse

- Utilizza una sola pagina con coppie <process id, numero di pagina>
- Però solo info su pagine in memoria principale
- Se page fault?

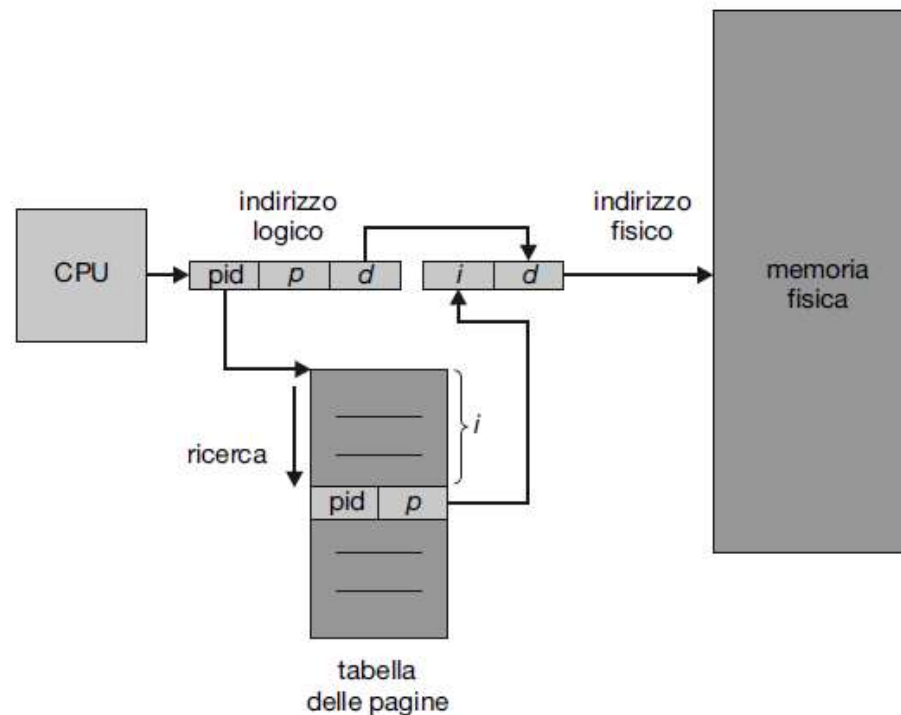


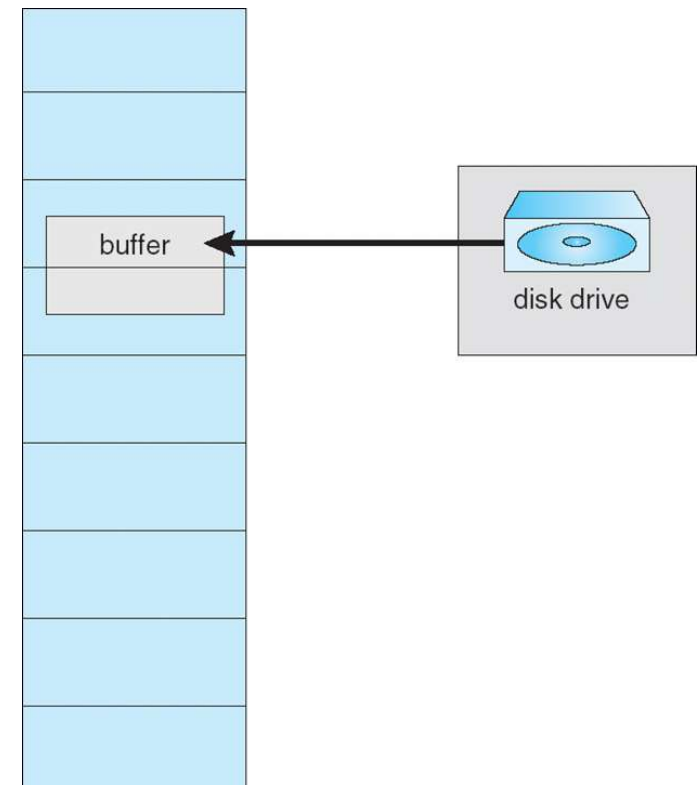
Figura 9.18 Tabella delle pagine invertita.

Altre Considerazioni – Tab Pagine inverse

- Utilizza una sola pagina con coppie <process id, numero di pagina>
- Però solo info su pagine in memoria principale
- Se page fault?
 - Non informazione complete su indirizzo logico della pagina referenziata
 - Demand paging richiede queste informazioni
 - Occorre leggerlo in una tabella anche questa paginata ...
 - Ogni processo ha una tabella delle pagine, ma serve solo sul page fault
 - ... ma anche la tabella può essere paginate e può fare page fault
 - Occorre una gestione attenta da parte del kernel

Altre Considerazioni – I/O interlock

- ❑ **I/O Interlock** – le pagine qualche volta devono essere “locked” in memoria
- ❑ Nel caso di I/O - pagine usate per copiare un file da un dispositivo (es. USB) devono essere locked per evitare che siano selezionate da un algoritmo di page replacement
 - ❑ Es. Processo richiede I/O, altro processo schedato, fa page fault e sottrae pagina in global replacement
- ❑ **Pinning** (fissare) pagine da tenere in memoria
- ❑ Lock bit per impedire il trasferimento di pagina
- ❑ Può essere usato anche durante il paging, es. per impedire che proc. ad alta priorità prenda pagine appena caricate a proc. a bassa priorità



Esempi di Sistemi Operativi

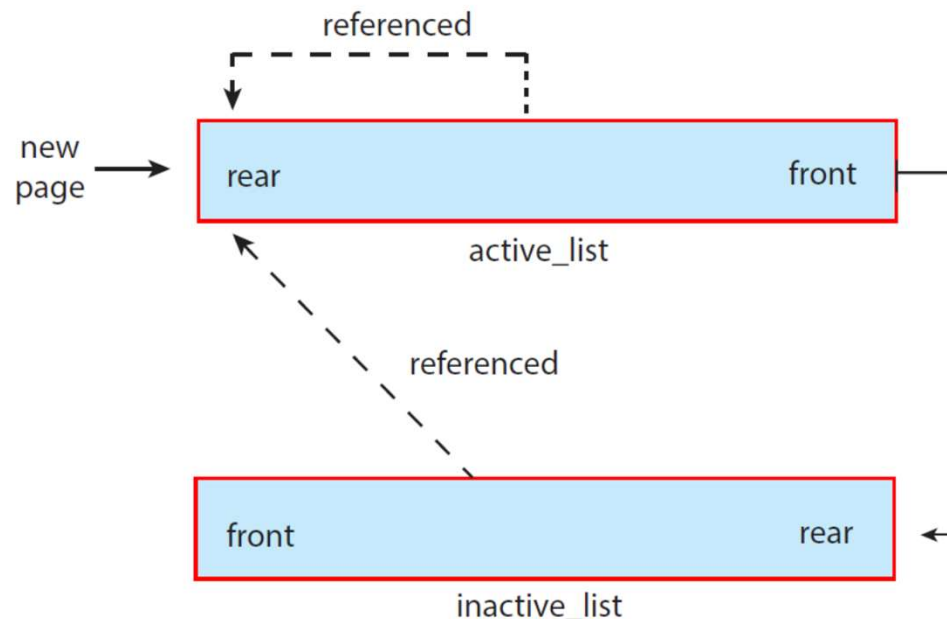
- Linux
- Windows
- Solaris

Linux

- ❑ Per memoria virtuale Linux usa demand paging con allocazione da lista di free frame
- ❑ Global page replacement con clock approx LRU
- ❑ Due liste di pagine **active_list** e **inactive_list**, le inactive sono elegibili per essere riusate

Linux

- Due liste di pagine **active_list** e **inactive_list**, le inactive sono elegibili per essere riusate
 - Un *accessed* bit per stabilire chi ha avuto un riferimento
 - Nuova pagina nel retro della active list (come ogni pagina con accesso in active)
 - Il bit è periodicamente resettato
 - Last Recently Used emerge sul fronte che retrocede nel retro dell'inactive
 - Un demone kswapd periodicamente verifica se occorre liberare memoria, se la memoria libera va sotto soglia scorre la inactive_list e libera frame



Windows

- ❑ Windows 10 supporta 32 o 64 bit (IA-32, IA-64, ARM)
- ❑ Windows 11 solo 64 bit
- ❑ Supporta demand paging, copy-on-write, paging, memory compression

Windows

- Usa demand paging con **clustering**.
 - Il clustering porta in memoria pagine “vicine” alla pagina che ha generato il page fault
 - Nel caso di data page il cluster è di 3 (la pagina richiesta, quella prima e dopo), altrimenti il cluster è 7
- Elemento cruciale è la gestione del working set

Windows

- Gestione del working set
 - Ai processi sono assegnati **working set minimum** e **working set maximum**
 - Working set minimum è il numero minimo di pagine che il processo è garantito avere in memoria (50 pagine)
 - Al processo possono essere assegnate tante pagine quanto è il suo working set maximum (345 pagine)
 - I valori possono essere ignorati a meno che non è settato **hard working set limit**
 - Win utilizza un LRU approx. clock con global e local replacement
 - Quando la memoria è sufficiente procede allocando free memory per servire il page fault
 - Quando la quantità di free memory nel sistema va sotto una soglia **automatic working set trimming** è eseguito per recuperare la quantità di free memory
 - Working set trimming rimuove pagine dai processi che hanno pagine in eccesso rispetto al loro working set minimum
 - Preferiti processi grandi e non attivi rispetto a quelli piccolo e attivi
 - Continua finché non si raggiunge la soglia (anche sottraendo sotto il min ws)
 - Trimming sia su processi utente che di sistema

Solaris

- ❑ Mantiene una lista di free pages da assegnare ai processi richiedenti
- ❑ **Lotsfree** – parametro soglia (quantità di free memory) per iniziare il paging (1/64 della dimensione della memoria fisica)
- ❑ **Desfree** – parametro soglia di free memory per incrementare il paging
- ❑ Scansione 4 volte al secondo per verificare la soglia,
- ❑ Se sotto soglia **Lotsfree** inizia il processo di **pageout**

- ❑ **Pageout** scansiona le pagine usando un algoritmo second chance modificato
 - ❑ **Front hand**: scansione delle pagine azzerando il reference bit
 - ❑ **Back hand**: assegna le pagine non referenziate alla free list e la copia in backing store
 - ❑ La distanza tra front e back è di **handspcard**
 - ❑ Una pagina non riassegnata ma con riaccesso può essere recuperate

Solaris

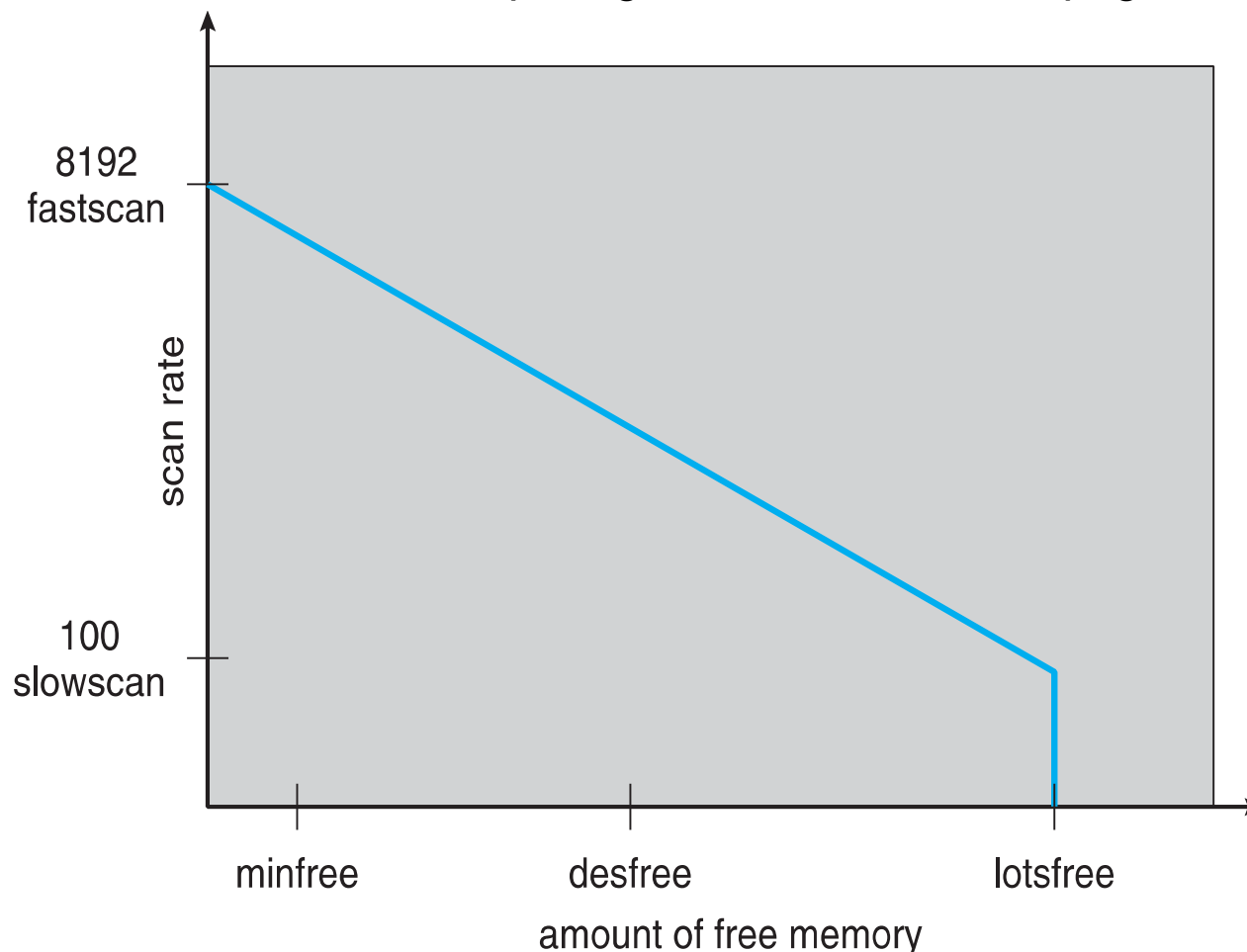
- **Scanrate** è la frequenza di scansione delle pagine.
 - Varia da **slowscan** a **fastscan**
 - ... da 100 pagine per secondo a $\text{mem-fisica}/2$ per secondo

- **Pageout** è chiamata più frequentemente in dipendenza dall'ammontare della free memory disponibile

- Il tempo di scansione dipende da **scanrate** e **handspeard** ...
 - Se **scanrate** è 100 pagine al secondo e **handspeard** è 1024 allora possono passare 10 secondi tra front hand e back hand ...
 - Nei sistemi reali il tempo tra front e back hand è di pochi secondi

Solaris

- ❑ **Pageout** fa il check di memoria 4 volte al secondo, se **desfree** è sotto soglia incrementa a 100 volte al secondo
- ❑ Se il SO non riesce a tenere **desfree** (per 30 sec) allora inizia lo swapping e libera i frame dei processi swappati
- ❑ Sotto **minfree** fa **pageout** per ogni richiesta di nuova pagina



Non fa swapping di shared pages, distingue tra pagine per processo e pagine per dati